МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Построение и анализ алгоритмов»

Студент гр. 4343 _____ Едина А.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n – количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j. Если $graph[i][j] = 0$ и ($i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Sample Input 1:

5

0 1 13 23 7

12 0 15 18 28

21 29 0 33 28

23 19 34 0 38

5 40 7 39 0

Sample Output 1:

78

0 4 2 3 1 0

Sample Input 2:

3

0 1 0

1 0 1

0 1 0

Sample Output 2:

no path

Вариант 7

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik. Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

Выполнение работы.

В основе точного решения лежит метод динамического программирования с мемоизацией. Состояние определяется парой (current_city, visited_mask), где: current_city – город, в котором мы находимся в данный момент; visited_mask – битовая маска посещённых городов (1 – посещён, 0 – нет). Рекурсивная функция visit(current_city, visited_mask) возвращает минимальную стоимость завершения маршрута (включая возврат в 0) и сам маршрут в виде списка городов (без текущего). Базовый случай – когда visited_mask == (1<<n)-1 (все города посещены). В этом случае возвращается стоимость ребра из current_city в 0, если оно существует, иначе – бесконечность. Для ускорения используется декоратор @lru_cache(None), который кеширует результаты вызовов. Временная сложность алгоритма – $O(n^2 \cdot 2^n)$, ёмкость памяти –$O(n \cdot 2\,n)$ для хранения кеша. При $n \leq 15$ это приемлемо (до ~500 тыс. состояний).

Алгоритм АЛШ1 является жадным: он строит маршрут, последовательно добавляя города, начиная с города 0. На каждом шаге, кроме последнего, для каждого непосещённого города-кандидата вычисляется значение $s + L$, где: s – вес ребра из текущего города в кандидата; L – эвристическая оценка минимальной стоимости прохода через оставшиеся непосещённые города. Эвристика L вычисляется как среднее арифметическое минимального входящего и минимального исходящего ребра для множества непосещённых городов (включая кандидата). Если для какого-то непосещённого города нет входящего или исходящего ребра, $L = \infty$. Город с минимальным значением $s + L$ выбирается следующим. Последний город добавляется без вычисления эвристики, так как выбора не остается. В конце проверяется возможность замыкания цикла в город 0. Если на каком-то шаге нет доступных городов или замыкание невозможно, алгоритм возвращает None. Сложность АЛШ1 – $O(n^3)$: на каждом из n шагов перебирается до n кандидатов, для каждого вычисление L требует $O(n)$ операций.

Для независимого тестирования был разработан скрипт generate.py, который создаёт матрицу весов заданного размера с возможностью выбора симметричного или произвольного заполнения. Веса генерируются случайно в диапазоне от 1 до 100. Матрица сохраняется в текстовый файл в формате, совместимом с основной программой.

Исследование

Точный алгоритм

Пространственная сложность

Алгоритм использует мемоизацию результатов рекурсивных вызовов функции visit(current_city, visited_mask). Количество возможных состояний:

- Маска посещённых городов: 2^n вариантов
- Текущий город: n вариантов.

Итого: $O(n2^n)$

Временная сложность

Для каждого состояния (current_city, visited_mask) алгоритм перебирает всех возможных следующих городов ($O(n)$ операций). Общее количество вызовов функции visit равно количеству достижимых состояний: $n \cdot 2^n$. Таким образом, временная сложность:

$$O(n * n \cdot 2^n) = O(n^2 * 2^n)$$

Итого: $O(n^2 * 2^n)$, что намного лучше полного перебора O(n!), но всё ещё экспоненциальная.

Приближенный алгоритм (АЛШ1)

Пространственная сложность

Алгоритм не использует мемоизацию и хранит только:

- Массив visited длины n (булевы значения);
- текущий путь path (максимум $n + 1$ целых чисел);

Дополнительная память не требуется. Итого сложность по памяти: $O(n)$

Временная сложность

Алгоритм строит маршрут за $n - 1$ шаг (выбор следующего города). На каждом шаге (кроме последнего) выполняется:

- перебор всех непосещённых городов-кандидатов: до n итераций;
- для каждого кандидата вычисляется эвристика L, которая перебирает все непосещённые города ($O(n)$ операций).

Таким образом, один шаг требует $O(n^2)$ операций, а всего шагов $O(n)$. Итого: $O(n^3)$

Тестирование

В этом разделе тестируются, оба алгоритма. Выходные данные сокращены до конкретных результатов, без промежуточных шагов.

1) Входные данные:

5

0 10 15 20 25

10 0 35 25 30

15 35 0 30 20

20 25 30 0 15

25 30 20 15 0

Выходные данные:

РЕЗУЛЬТАТЫ АЛШ-1

Стоимость: 85.0

Путь: 0 1 3 4 2 0

РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА

85

0 1 3 4 2 0

2) Входные данные:

6

0 1 1 100 100 100

100 0 1 1 100 100

100 100 0 1 1 100

100 100 100 0 1 1

1 100 100 100 0 1

1 1 100 100 100 0

Выходные данные:

РЕЗУЛЬТАТЫ АЛШ-1

Стоимость: 204.0

Путь: 0 1 3 4 5 2 0

РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА

6

0 1 2 3 4 5 0

3) Входные данные:

5

0 1 0 0 0

1 0 1 0 0

0 1 0 1 0

0 0 1 0 1

0 0 0 1 0

Выходные данные:

РЕЗУЛЬТАТЫ АЛШ-1

пути нет

РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА

пути нет

4) Входные данные:

15

0 50 31 42 77 99 63 7 46 18 8 38 38 68 82

25 0 81 41 61 10 44 60 16 91 97 58 68 88 31

66 77 0 36 94 35 19 95 1 81 76 80 4 99 12

22 81 99 0 97 30 61 47 18 74 48 20 99 76 53

77 46 64 49 0 7 23 24 75 97 89 37 25 83 62

95 46 55 90 95 0 33 9 80 11 34 74 7 64 79

95 32 1 93 91 98 0 31 70 100 9 68 37 55 16

51 91 85 13 53 49 81 0 20 92 74 61 52 41 77

82 4 73 86 71 7 71 71 0 89 72 74 21 75 1

42 55 31 50 3 3 81 47 91 0 39 5 15 50 16

71 38 58 60 96 92 77 94 84 20 0 31 69 78 38

58 66 94 68 80 36 49 94 16 3 99 0 44 89 30

42 25 70 49 6 89 58 80 3 41 10 12 0 76 98

38 56 68 80 88 25 44 18 37 33 69 40 94 0 71

85 92 67 8 53 4 8 87 28 10 43 46 75 7 0

Выходные данные:

РЕЗУЛЬТАТЫ АЛШ-1

Стоимость: 433.0

Путь: 0 7 8 14 5 12 11 9 2 6 10 1 3 13 4 0

РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА

167

0 10 1 5 12 11 9 4 6 2 8 14 13 7 3 0

5) Входные данные:

5

0 25 58 39 80

18 0 99 8 16

74 28 0 77 23

41 7 80 0 74

98 2 8 53 0

Выходные данные:

РЕЗУЛЬТАТЫ АЛШ-1

Стоимость: 167.0

Путь: 0 1 4 2 3 0

РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА

132

0 2 4 1 3 0

Приложение

Код из lb2.py:

```
import sys
from functools import lru_cache

DEBUG = True
INF = 10**15

def log(message):
    if DEBUG:
        print(message)

def exact_dp(matrix):
    n = len(matrix)
    all_visited_mask = (1 << n) - 1

    @lru_cache(None)
    def visit(current_city, visited_mask):
        if visited_mask == all_visited_mask:
            return_cost = matrix[current_city][0]
            if return_cost == 0:
                log(f"нет пути из {current_city} в 0.")
                return INF, []

             log(f"возврат из {current_city} в 0. Вес: {return_cost},
маска {bin(visited_mask)}")
            return return_cost, [0]

        min_cost = INF
        best_path_suffix = []

        log(f"город={current_city}, маска={bin(visited_mask)}")

        for next_city in range(n):
            city_not_visited = not (visited_mask & (1 << next_city))
            path_exists = matrix[current_city][next_city] != 0

            if city_not_visited and path_exists:
                edge_cost = matrix[current_city][next_city]
                new_visited_mask = visited_mask | (1 << next_city)
                  log(f"попытка перейти: {current_city} -> {next_city}
(вес ребра {edge_cost})")
                    remaining_cost, remaining_path = visit(next_city,
new_visited_mask)
                total_cost = edge_cost + remaining_cost

                if total_cost < min_cost:
                    min_cost = total_cost
                    best_path_suffix = [next_city] + remaining_path
        log(f" {min_cost}, {best_path_suffix}")
        return min_cost, best_path_suffix

    optimal_cost, optimal_path_suffix = visit(0, 1)

    if optimal_cost == INF:
        return None, None
```

```
    full_path = [0] + optimal_path_suffix
    return optimal_cost, full_path

def get_heuristic_L(matrix, visited, current_city):
    n = len(matrix)
    min_enter = INF
    min_exit = INF

    for u in range(n):
        if not visited[u]:
            if matrix[u][current_city] != 0 and matrix[u][current_city]
< min_enter:
                min_enter = matrix[u][current_city]
            if matrix[current_city][u] != 0 and matrix[current_city][u]
< min_exit:
                min_exit = matrix[current_city][u]

    if min_enter == INF or min_exit == INF:
        return INF

    return (min_enter + min_exit) / 2.0

def alsh1(matrix):
    n = len(matrix)
    visited = [False] * n
    path = [0]
    visited[0] = True
    current_cost = 0.0
    current_city = 0

    log("\nАЛШ-1")

    while len(path) < n:
        best_value = INF
        best_next_city = -1
        best_edge_cost = 0

        if len(path) == n - 1:
            for i in range(n):
                if not visited[i]:
                    best_next_city = i
                    best_edge_cost = matrix[current_city][i]
                    break
        else:
            for candidate_city in range(n):
                                if   not   visited[candidate_city]   and
matrix[current_city][candidate_city] != 0:
                    s = matrix[current_city][candidate_city]

                    visited[candidate_city] = True
                   L = get_heuristic_L(matrix, visited, candidate_city)
                    visited[candidate_city] = False

                    value = s + L
                        log(f"кандиадт {candidate_city}: s={s}, L={L},
s+L={value}")
```

```
                    if value < best_value:
                        best_value = value
                        best_next_city = candidate_city
                        best_edge_cost = s

        if best_next_city == -1 or best_edge_cost == 0:
            log("нет доступных городов")
            return None, None

        path.append(best_next_city)
        visited[best_next_city] = True
        current_cost += best_edge_cost
        current_city = best_next_city
            log(f"- выбран город {current_city}, стоимость шага:
{best_edge_cost}")

    closing_cost = matrix[current_city][0]
    if closing_cost == 0:
        log("нет пути в начальный город")
        return None, None

    current_cost += closing_cost
    path.append(0)
    log(f"- замыкание цикла в 0, стоимость: {closing_cost}\n")

    return current_cost, path

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    matrix = []
    idx = 1
    for _ in range(n):
        row = []
        for _ in range(n):
            row.append(int(input_data[idx]))
            idx += 1
        matrix.append(row)

    cost, path = exact_dp(matrix)

    if DEBUG:
        approx_cost, approx_path = alsh1(matrix)
        print("РЕЗУЛЬТАТЫ АЛШ-1")
        if approx_cost is None:
            print("пути нет\n")
        else:
            print(f"Стоимость: {approx_cost}")
            print(f"Путь: {' '.join(map(str, approx_path))}\n")
        print("РЕЗУЛЬТАТЫ ТОЧНОГО АЛГОРИТМА")

    if cost is None:
        print("пути нет")
    else:
        print(cost)
```

```
        print(" ".join(map(str, path)))

if __name__ == "__main__":
    main()
```

Код из generate.py:

```
import random

def gen(filename="test.txt"):
    n = int(input("колво городв: "))
    symmetric = int(input("симметричная? (1/0): ")) == 1
    filename = input("Имя файла для сохранения: ") or filename

    matrix = [[0] * n for _ in range(n)]

    if symmetric:
        for i in range(n):
            for j in range(i + 1, n):
                w = random.randint(1, 100)
                matrix[i][j] = w
                matrix[j][i] = w
    else:
        for i in range(n):
            for j in range(n):
                if i != j:
                    matrix[i][j] = random.randint(1, 100)

    with open(filename, 'w') as f:
        f.write(str(n) + "\n")
        for row in matrix:
            f.write(" ".join(map(str, row)) + "\n")

    print(f"сохранили в {filename}")

def main():
    gen()

if __name__ == "__main__":
    main()
```